

TP JDBC : Connexion Base de Données

Matisse
Kra
bts sio
Sommaire

Objectif du TP	2
Configuration de la Base de Données	2
Création de la classe Livre et "Intelligence" JDBC	2
Manipulation des données (Main)	3
La Vue : Synchronisation (Accueil.java)	4
Difficultés rencontrées et Solutions	5
Conclusion	5

Objectif du TP

L'objectif de ce TP était de comprendre comment interagir avec une base de données relationnelle (MySQL) depuis un programme Java. Nous avons utilisé le pilote **JDBC** pour établir la connexion, exécuter des requêtes SQL (SELECT, INSERT, UPDATE) et interfacer ces données avec une interface graphique Swing.

Configuration de la Base de Données

Nous avons d'abord préparé l'environnement en activant le serveur MySQL et en intégrant le pilote JDBC au projet. Une base de données nommée `slam2_td2` et une table `Livre` ont été créées.

ISBN	titre ▲ 1	auteur	prix
4293996815	Absalon Absalon !	William Faulkner	29.5
5979656845	Berlin Alexanderplatz	Alfred Doblin	30
6188391185	Cent ans de solitude	Gabriel Garcia Marquez	6
978-2-PRO-JAVA	Coder comme un pro en Java	M.Gravouil	99
1317442277	Contes	Hans Christian Andersen	10
6154636942	Crime et Chatiment	Fiodor Dostoievski	30
5879805162	Decameron	Boccace	16
4548950963	Divine Comedie	Dante Alighieri	25
5147239320	Don Quichotte	Miguel de Cervantes	15

Création de la classe Livre et "Intelligence" JDBC

- Nous avons développé une classe Livre qui ne se contente pas de stocker des données en mémoire, mais qui synchronise ses modifications avec la base de données.
- Pour cela, nous avons modifié les Setters (mutateurs). Par exemple, la méthode `setPrix` prend désormais en paramètre un objet `Statement` pour exécuter une requête UPDATE immédiatement après la modification de l'objet.

```
// 4. SETTERS JDBC (Mutateurs avec mise à jour BDD)

//Set ISBN
public void setISBN(String isbn, Statement stmt) {
    this.isbn = isbn;
    try {
        String requete = "UPDATE Livre SET ISBN = '" + isbn + "' WHERE titre = '" + this.titre + "'";
        stmt.executeUpdate(requete);
        System.out.println(x: "Base de données mise à jour : Nouvel ISBN enregistré."); // Replace this
    } catch (SQLException e) {
        System.out.println("Erreur SQL : " + e.getMessage()); // Define a constant instead of duplicat
    }
}

//Set Titre
public void setTitre(String titre, Statement stmt) {
    this.titre = titre;
    try {
        String requete = "UPDATE Livre SET titre = '" + titre + "' WHERE ISBN = '" + this.isbn + "'";
        stmt.executeUpdate(requete);
        System.out.println(x: "Base de données mise à jour : Nouveau titre enregistré."); // Replace th
    } catch (SQLException e) {
        System.out.println("Erreur SQL : " + e.getMessage()); // Replace this use of System.out by a l
    }
}
```

Manipulation des données (Main)

Fonction "Mise à jour en direct" Grâce à la modification des setters, si le programme modifie le prix d'un livre (ex: le livre ISBN "1317442277" passant à 10€), le changement est instantanément répercuté dans la base MySQL sans action manuelle supplémentaire.

Fonction "Ajout Persistant" L'ajout du livre "Coder comme un pro en Java" de M. Gravouil a permis de tester le nouveau constructeur. Le livre est ajouté à la liste visuelle et inséré dans la table SQL simultanément.

```
/**
 * Constructeur "Intelligent"
 * Crée l'objet et l'insère directement dans la BDD via le Statement.
 */
public Livre(String isbn, String titre, String auteur, int prix, Statement stmt) {
    // A. Initialisation de l'objet Java
    this.isbn = isbn;
    this.titre = titre;
    this.auteur = auteur;
    this.prix = prix;

    // B. Insertion SQL directe
    try {
        String requete = "INSERT INTO Livre (ISBN, titre, Auteur, prix) VALUES ('"
            + isbn + "', '"
            + titre + "', '"
            + auteur + "', '"
            + prix + "')";

        stmt.executeUpdate(requete);
        System.out.println("Succès : Le livre " + titre + " a été ajouté à la BDD !")
    } catch (SQLException e) {
        System.out.println("Erreur lors de l'insertion en BDD : " + e.getMessage());
    }
}
```

Dans le programme principal, nous avons testé la modification et l'insertion de données :

1. Mise à jour (UPDATE) : Modification du prix du livre (ISBN 1317442277) pour le passer à 10€.
2. Insertion (INSERT) : Création d'un constructeur "intelligent" permettant d'instancier un livre (Ex: "Coder comme un pro en Java" de M. Gravouil) et de l'insérer directement en base.

```
// B. Configuration de la connexion JDBC
String url = "jdbc:mysql://localhost:3306/slam2_td2";
String login = "root";
String passwd = ""; // Pas de mot de passe par défaut sur XAMPP

try {
    // 1. Établissement de la connexion
    Connection con = DriverManager.getConnection(url, login, passwd);    Add password protection to this database. [+1 location]
    Statement stmt = con.createStatement();

    // QUESTION 7 : MODIFICATION D'UN PRIX (UPDATE)

    Livre livreAmodifier = new Livre(isbn: "1317442277", titre: "Titre temporaire", auteur: "Auteur temporaire", prix: 0);

    // Appel de la méthode intelligente setPrix avec Statement
    livreAmodifier.setPrix(prix: 10, stmt);
    System.out.println(x: "-> Le prix a été modifié à 10€ pour l'ISBN 1317442277.");    Replace this use of System.out by a logger.

    // QUESTION 9 : AJOUT DU LIVRE M. GRAVOUIL (INSERT)

    int prixEntier = (int) 99.50;

    // Création + Insertion BDD via le constructeur "Intelligent" (créé à la question 8)
    Livre livrePro = new Livre(isbn: "978-2-PRO-JAVA", titre: "Coder comme un pro en Java", auteur: "M.Gravouil", prixEntier, stmt);

    // Ajout à la liste
    listeDesLivres.add(livrePro);
    System.out.println(x: "-> Livre ajouté avec succès à la liste et à la BDD.");    Replace this use of System.out by a logger.

    // 2. Fermeture
    stmt.close();
    con.close();
} catch (SQLException e) {
    e.printStackTrace();
}

return listeDesLivres;
```

```
-> Le prix a été modifié à 10€ pour l'ISBN 1317442277.
Succès : Le livre Coder comme un pro en Java a été ajouté à la BDD !
-> Livre ajouté avec succès à la liste et à la BDD.

--- CONTENU FINAL DE LA BIBLIOTHEQUE ---
Taille de la bibliotheque: 5
Livre: Titre 1 (11111) - 1.0€
Livre: Titre 2 (22222) - 2.0€
Livre: Titre 3 (33333) - 3.0€
Livre: Titre 4 (44444) - 4.0€
Livre: Coder comme un pro en Java (978-2-PRO-JAVA) - 99.0€
```

La Vue : Synchronisation (Accueil.java)

Dans la fenêtre principale, nous avons supprimé la création manuelle des livres.

- Chargement dynamique : Au lancement, le programme exécute un `SELECT * FROM Livre`.
- Traitement : Une boucle `while(rs.next())` parcourt les résultats (le `ResultSet`) pour reconstruire la liste `al` avec les vraies données de la base.

```
// --- INITIALISATION DONNÉES (BDD) ---
String url = "jdbc:mysql://localhost:3306/slam2_td2";
String login = "root";
String passwd="";

try {
    Class.forName(className: "com.mysql.cj.jdbc.Driver"); // Driver récent
    Connection con = DriverManager.getConnection(url,login,passwd); // Add password protection to this database. [+1 locat
    Statement stmt = con.createStatement(); // Use try-with-resources or close this "Statement" in a "finally" clause.

    // ATTENTION : J'ai commenté cette ligne car elle fait planter le programme
    // si l'ISBN 999 existe déjà. C'était juste pour un test.
    // String sql = "INSERT INTO `livre` (`ISBN`, `TITRE`, `AUTEUR`, `PRIX`) VALUES ('999', 'test', 'Auteur test', 10); ";
    // stmt.executeUpdate(sql);

    // Lecture de la BDD
    String requete = "SELECT * FROM Livre"; // Don't use the query "SELECT *".
    ResultSet rs = stmt.executeQuery(requete);

    while (rs.next()) {
        String isbn = rs.getString(columnLabel: "ISBN");
        String titre = rs.getString(columnLabel: "titre");
        String auteur = rs.getString(columnLabel: "Auteur");

        int prix = rs.getInt(columnLabel: "prix");

        // On crée l'objet Java et on l'ajoute à la liste
        Livre livreBdd = new Livre(isbn, titre, auteur, prix);
        al.add(livreBdd);
    }

    rs.close();
    stmt.close();
    con.close();
} catch (Exception e) {
    e.printStackTrace();
    JOptionPane.showMessageDialog(parentComponent: null, "Erreur de connexion BDD : " + e.getMessage());
}
```

Difficultés rencontrées et Solutions

Durant la réalisation du TP, quelques points techniques ont demandé une attention particulière :

Gestion des types de données

- Problème : Le livre à ajouter avait un prix de 99.50€ (double), mais notre classe Livre attendait un entier (int).
- Solution : Nous avons effectué un "cast" explicite (int) 99.50 lors de l'appel du constructeur pour adapter la donnée.

Syntaxe SQL dans Java

- Problème : Les erreurs de syntaxe SQL lors de la concaténation des chaînes de caractères (oubli des apostrophes autour des titres).
- Solution : Vérification rigoureuse des requêtes, par exemple : "VALUES ('" + titre + "', ..." pour s'assurer que MySQL reçoive bien les guillemets nécessaires aux chaînes de caractères.

Conclusion

Ce TP a permis de comprendre le rôle central du JDBC dans les applications d'entreprise. Nous avons appris à distinguer les méthodes executeQuery (pour lire des données) et executeUpdate (pour modifier/insérer des données). L'application est maintenant capable de conserver ses données entre deux redémarrages, ce qui est une étape essentielle dans le développement logiciel.